CREDIT TRUST: A DIGITAL CREDIT TRANSPRANCY SYSTEM FOR LOCAL RETAIL SHOPS

A

MAJOR PROJECT-II REPORT

Submitted in partial fulfillment of the requirement.

for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

By

GROUP NO.17

Shruti Tiwari	**0187CS221189**
Siya Gajbhiye	**0187CS221192**
Sitvat Fatima	**0187CS221191**

Under the guidance of

Dr. Komal Tahiliani

(Associate Professor)

Department of Computer Science & Engineering
Sagar Institute of Science & Technology (SISTec), Bhopal (M.P)

Approved by AICTE, New Delhi & Govt. of M.P.
Affiliated to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.)

June - 2026

Sagar Institute of Science & Technology (SISTec), Bhopal (M.P)

Department of Computer Science & Engineering

CERTIFICATE

We hereby certify that the work which is being presented in the B.Tech. Major Project-II Report entitled **CREDIT TRUST: A DIGITAL CREDIT TRANSPARENCY SYSTEM FOR LOCAL RETAIL SHOPS,** in partial fulfillment of the requirements for the award of the degree of ***Bachelor of Technology,*** submitted to the Department of **Computer Science & Engineering**, Sagar Institute of Science & Technology (SISTec)**,** Bhopal (M.P.) is an authentic record of our own work carried out during the period from Jan-2026 to Jun-2026 under the supervision of **Dr. Komal Tahiliani.**

The content presented in this project has not been submitted by me for the award of any other degree elsewhere.

Shruti Tiwari	**Siya Gajbhiye**	**Sitvat Fatima**
0187CS221189	**0187CS221192**	**0187CS221191**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Dr. Komal Tahiliani	**Prof. Nargish Gupta**	**Dr. Manish Billore**
Project Guide	**HOD, CSE**	**Principal**

ACKNOWLEGMENT

We take this occasion to thank God, almighty for blessing us with his grace and taking our endeavor to a successful culmination. We extend our sincere and heartfelt thanks to our esteemed Project Guide, **Dr Komal Tahiliani,** for providing us with the right guidance and advice at crucial junctures and for showing us the right way.

We would like to express our sincere thanks to **Dr. Manish Billore, Principal, SISTec**, Gandhi Nagar, Bhopal for giving us an opportunity to undertake this project.

We would like to express our sincere thanks to **Dr. Swati Saxena, Vice Principal, SISTec**, Gandhi Nagar, Bhopal, for giving us an opportunity to undertake this project.

We also take this opportunity to express a deep sense of gratitude to **Prof. Nargish Gupta (HOD**) of the Department of Computer Science & Engineering for his kindhearted support. I would like to thank all those people who helped me directly or indirectly to complete.my project whenever I found myself in problems. All our faculties encouraged me and due to their kindness and helpful nature and help I got very much confidence in completing this project.

I am deeply inherited who devoted his precious time to giving me the information about the various aspects and giving support and guidance at every point of time. I am thankful to their kind and supportive nature. His inspiring nature has always made my work easy.

TABLE OF CONTENTS

ABSTRACT

The project **"Credit Trust: A Digital Credit Transparency System for Local Retail Shops"** is a smart digital solution designed to improve credit (udhari) management for small retail businesses. In today's rapidly growing digital economy, India has witnessed a significant rise in digital transactions, highlighting the need for efficient and transparent financial systems. However, many local shopkeepers still rely on traditional paper-based methods, which lead to errors, lack of transparency, and difficulty in tracking payments. This system aims to digitize credit records by providing a mobile-based platform that allows shopkeepers to manage customer details, record transactions, track payments, and store supplier bills securely using cloud technology like Firebase. It also enables customers to view their credit history and receive real-time updates, ensuring transparency and reducing disputes. By replacing manual processes with automated digital solutions, the system improves accuracy, efficiency, and trust between shopkeepers and customers. The project aligns with **Sustainable Development Goal (SDG) 8: Decent Work and Economic Growth** by promoting digital adoption among small businesses and enhancing financial management practices. It supports the transition towards cashless and transparent transactions, contributing to economic growth and improved business operations. Thus, the system bridges the gap between traditional credit management and modern digital solutions, empowering local vendors with a reliable, scalable, and user-friendly platform.

LIST OF ABBREVIATIONS

ACRONYM	FULL FORM
DB	Database
IDE	Integrated Development Environment
NoSQL	Non-Structured Query Language (used in Firebase/MongoDB)
SDK	Software Development Kit
URL	Uniform Resource Locator

LIST OF FIGURES

Chapter 1
Introduction

CHAPTER 1
INTRODUCTION

1.1 ABOUT PROJECT

The traditional system of managing credit (udhari) in local shops is mostly manual and prone to errors, data loss, and lack of transparency. Shopkeepers rely on paper registers, which often lead to miscalculations and difficulty in tracking payments. To solve these issues, our project "**Credit Trust: A Digital Credit Transparency System for Local Retail Shops**" provides a smart digital solution. It is a mobile-based application that allows shopkeepers to manage customers, record transactions, and store supplier bills securely. The system uses Firebase for safe and real-time data storage. It includes modules like Admin, Customer, Credit Management, and Supplier Billing. Customers can also view their credit records and report any discrepancies. This ensures transparency and builds trust between shopkeepers and customers. The system replaces manual work with an efficient, accurate, and easy-to-use digital platform.

1.2 PURPOSE

The purpose of the **Digital Credit Transparency System for Local Retail Shops (Credit Trust)** is to create a user-centric platform that helps shopkeepers manage credit (udhari) records efficiently. It uses a mobile application with Firebase to provide secure and real-time tracking of transactions. The system reduces errors and replaces traditional paper-based record keeping with a digital solution. It also allows customers to view their credit history and report any discrepancies, ensuring transparency and trust. By digitizing supplier bills and records, it minimizes data loss and mismanagement. Overall, the platform improves efficiency and supports digital transformation for small local businesses.

1.3 PROJECT OBJECTIVE

- **Manage Credit Records Efficiently:** Enable shopkeepers to digitally record and track udhari (credit) transactions accurately, reducing manual errors and improving record management.
- **Develop a User-Friendly Application:** Design a simple and intuitive mobile interface so that shopkeepers and customers can easily use the system without technical difficulty.
- **Ensure Smooth System Integration:** Implement Firebase integration to allow seamless data storage, real-time updates, and reliable system performance.

- **Facilitate Secure Data Management:** Store customer details, credit records, and supplier bills securely in the cloud to prevent data loss and mismanagement.
- **Promote Transparency and Trust:** Allow customers to view their credit history and raise issues, ensuring clear communication and trust between shopkeeper and customer.
- **Enhance Accessibility and Efficiency:** Provide an easy-to-use and scalable solution for local vendors, helping them shift from manual registers to a digital system.

1.4 SIGNIFICANCES OF PROJECT

- **Improved Transparency in Credit System:** The system provides clear and accurate digital records of udhari transactions, helping both shopkeepers and customers maintain trust and avoid disputes.
- **Bridging the Digital Gap for Local Vendors:** It replaces traditional paper-based systems with an easy-to-use digital platform, especially beneficial for small shopkeepers with limited technical resources.
- **Better Decision-Making and Management:** Shopkeepers can track payments, manage supplier bills, and analyze records efficiently, leading to improved financial decisions and business growth.
- **Secure, Scalable, and Accessible Solution:** The mobile-based platform ensures safe data storage, easy access, and usability for a wide range of users, promoting digital adoption in local retail businesses.

Chapter 2
Software and Hardware Requirements

CHAPTER 2
SOFTWARE & HARDWARE REQUIREMENTS

2.1 HARDWARE REQUIREMENTS

For the Credit Trust application, the following hardware specifications are required to ensure smooth performance, efficient data handling, and seamless user interaction.

- **Processor:** Intel Core i3 or higher / AMD equivalent to handle application processing and backend operations efficiently.
- **RAM:** Minimum 4 GB (8 GB recommended) to support smooth multitasking and real-time data updates.
- **Storage:** Minimum 128 GB SSD for faster data access, storage of customer records, transaction history, and application files.
- **Network:** Stable internet connection to enable real-time data synchronization, cloud database access, and seamless communication between frontend and backend services.
- **Device:** Android/IOS smartphone or desktop system to access and operate the application conveniently for daily shop management.

2.2 SOFTWARE REQUIREMENTS

2.2.1 DEVELOPMENT

- **Operating System (OS):** Windows 10/11, macOS, or Linux
- **Programming Languages:** Dart (Flutter)
- **Frameworks/Libraries:** Flutter SDK, Firebase SDK
- **Tools:** VS Code / Android Studio, Git/GitHub

2.2.2 BACKEND FUNCTIONALITY

- **Backend as a Service (BaaS):** Firebase for authentication and management of admin and customer data.
- **Database:** MongoDB Atlas for storing and managing product stock and inventory data.
- **Cloud Storage:** Cloudinary for storing and managing images and media files.

2.2.3 DEPLOYMENT

- **Platform:** Android (APK) and Web (optional)
- **Cloud Services:** Firebase services for hosting, authentication, and real-time data handling.

- **Database Hosting:** MongoDB Atlas for scalable cloud-based database management.

2.2.4 AI INTEGRATION

- **Voice Assistance:** Utilized for text-to-speech functionality to announce daily pending dues and provide audio-based updates to the admin.
- **Smart Insights:** Basic intelligent logic to analyze sales, recovery, and customer dues for better decision-making.

2.3 CLIENT-SIDE REQUIREMENTS

- **Device Compatibility:** Android smartphone, tablet, or desktop/laptop to access and operate the application efficiently.
- **Internet Connection:** Stable internet connection for real-time data synchronization with Firebase, MongoDB Atlas, and Cloudinary services.

Chapter 3
Problem Description

CHAPTER 3
PROBLEM DESCRIPTION

In today's fast-growing digital era, many local shopkeepers face challenges in managing credit (udhari) records efficiently using traditional paper-based methods. These manual systems often lead to calculation errors, misplaced entries, and difficulty in tracking payments. Shopkeepers also struggle with managing supplier bills and maintaining proper financial records. Due to lack of transparency, disputes can arise between customers and shopkeepers regarding credit details. Additionally, paper records are at high risk of damage or loss, causing permanent data issues. Many small vendors lack access to simple digital solutions that can streamline their operations. Hence, there is a strong need for **A Digital Credit Transparency System for Local Retail Shops** that helps shopkeepers manage records accurately, ensure transparency, and improve overall efficiency in their business operations.

3.1 CURRENT SCENARIO AND CHALLENGES

In recent years, the need for digital solutions in local shops has increased due to growing transactions and record-keeping demands. However, many shopkeepers still rely on paper-based systems for managing credit (udhari). These methods are time-consuming, error-prone, and lack transparency. Tracking payments and managing records becomes difficult and unreliable. Therefore, several challenges still exist in ensuring efficient and secure credit management for local vendors.

- **Lack of Proper Record Management**: Many shopkeepers struggle to maintain accurate credit (udhari) records using paper registers, leading to errors and confusion.

- **Resource Constraints**: Small vendors often lack access to affordable digital tools, making it difficult to shift from manual to digital systems.

- **Delayed Payment Tracking**: Without proper systems, tracking pending payments becomes slow and inefficient, affecting cash flow.

- **Awareness Gap**: Many shopkeepers are unaware of simple digital solutions that can improve transparency, efficiency, and overall business management.

3.2 IMPORTANCE OF AUTOMATION

Automation plays a vital role in transforming the **Credit Trust: A Digital Credit Transparency System for Local Retail Shops** by making credit management more efficient, accurate, and reliable for local shopkeepers. With the use of digital technologies like mobile applications and cloud databases (Firebase), automation helps overcome the limitations of traditional paper-based record keeping. The key importance of automation in this system includes:

- **Streamlined Credit Management:** Automation enables real-time recording and tracking of udhari (credit) transactions, eliminating the need for manual entries and reducing calculation errors.
- **Automated Data Storage:** All customer details, credit records, and supplier bills are stored securely in the cloud, ensuring easy access and preventing data loss or misplacement.
- **Real-Time Updates and Transparency:** The system automatically updates records, allowing both shopkeepers and customers to view accurate transaction details anytime, improving transparency.
- **Reduced Human Error:** Manual record keeping often leads to mistakes and inconsistencies, whereas automation ensures accurate and standardized data management.
- **Enhanced Efficiency and Scalability:** Automation allows shopkeepers to manage multiple customers and transactions efficiently, making the system scalable for growing businesses.
- **Time and Cost Effectiveness:** By reducing manual work and paperwork, automation saves time, effort, and operational costs, improving overall productivity of local vendors.

3.3 SCOPE OF THE PROJECT

The **Digital Credit Transparency System for Local Retail Shops (Credit Trust)** aims to provide an automated, user-friendly platform for efficient credit (udhari) management for local shopkeepers. Key aspects include

- **Digital Record Management:** The system enables shopkeepers to maintain all credit (udhari) records digitally, replacing traditional paper registers with a more reliable solution.
- **Customer and Transaction Handling:** It allows easy addition of customers and efficient tracking of their credit transactions and payment history.

- **Secure Cloud Storage:** All data including customer details and supplier bills are stored securely using cloud technology, ensuring safety and easy access.
- **Transparency and Trust:** Customers can view their credit records anytime, which helps in reducing disputes and building trust with shopkeepers.
- **Scalability for Small Businesses:** The system can handle multiple customers and transactions, making it suitable for growing local businesses.
- **Future Expansion Capability:** The project can be extended with features like payment reminders, digital payments, and analytics for better business management.

3.4 SUMMARY

The **Digital Credit Transparency System for Local Retail Shops (Credit Trust)** helps local shopkeepers efficiently manage credit (udhari) records through automation and digital technology. It replaces traditional paper-based systems with a mobile application that enables accurate recording, tracking, and management of customer transactions and supplier bills. By using cloud technology (Firebase), the system ensures secure data storage, real-time updates, and easy accessibility for both shopkeepers and customers. It promotes transparency by allowing customers to view their credit history and report discrepancies, building trust and reducing conflicts. With a simple and user-friendly interface, the platform supports small vendors in improving efficiency, reducing errors, and saving time. Ultimately, it serves as a smart, reliable, and scalable solution that drives digital transformation and better financial management in local retail businesses.

Chapter 4
Literature Survey

CHAPTER 4
LITERATURE SURVEY

India is home to the world's second-largest smartphone market, and the rapid expansion of internet connectivity has significantly contributed to the growth of digital payments across the country. With approximately 504 million people regularly using the internet, the foundation for a digital economy has become stronger over time. The entry of affordable data services, especially through Reliance Jio, played a transformative role in increasing internet accessibility even in semi-urban and rural areas. Digital wallets and online payment applications have enabled users to store money, payment details, and other financial information securely, making transactions faster and more convenient. However, despite these advancements, challenges such as low internet speed, poor network quality, and unequal digital infrastructure remain major barriers that limit the widespread success of digital payments in India. [1].

Over the years, India has witnessed a steady shift toward a cashless economy, with digital payment systems gaining popularity among consumers and businesses alike. Many studies suggest that people increasingly prefer electronic payment methods because they reduce the risks associated with carrying cash and help minimize corruption, tax evasion, and other illegal activities. Digital transactions also lower the cost of printing, handling, and monitoring physical currency, thereby improving overall economic efficiency. Although the COVID-19 pandemic temporarily slowed down digital payments in certain sectors, it also accelerated their adoption in others, such as online gaming, e-commerce, and utility payments. This shift highlights the flexibility and resilience of digital payment systems in adapting to changing economic conditions and consumer behavior. [2].

The adoption of digital payments is influenced by multiple factors, including ease of use, convenience, security, and trust. Users are more likely to adopt digital payment systems if they find them simple to operate and reliable in terms of transaction processing. Technological advancements such as big data, artificial intelligence, and the Internet of Things (IoT) have further enhanced the efficiency and user experience of digital platforms. These technologies allow users to perform transactions anytime and anywhere, increasing flexibility and accessibility. However, concerns regarding data privacy, cybersecurity threats, and unauthorized access to financial information continue to pose challenges. Trust remains a critical factor, as any security breach can significantly affect user confidence and slow down adoption rates. [3].

Demographic and socioeconomic factors also play an important role in determining the acceptance of digital payments in India. Variables such as education level, age, income, and digital literacy strongly influence how individuals perceive and use digital payment systems. In rural areas, lack of awareness, poor infrastructure, and limited access to smartphones and internet services act as major obstacles. Government initiatives, along with support from banks and fintech companies, are essential in addressing these challenges. Efforts such as financial literacy programs, incentives like cashback offers, and the introduction of secure and user-friendly platforms have helped encourage more users to shift toward digital transactions. Additionally, government-backed systems and policies have contributed to building trust and promoting the adoption of cashless payment methods. Looking toward the future, emerging technologies such as blockchain, artificial intelligence, and cryptocurrencies are expected to further revolutionize the digital payment landscape. These innovations have the potential to enhance security, transparency, and efficiency in financial transactions while reducing dependency on traditional banking systems. Government regulations and active involvement will be crucial in ensuring a secure and stable digital ecosystem. As infrastructure improves and awareness continues to grow, India is steadily moving toward a more inclusive, transparent, and efficient cashless economy that can support long-term economic development. [4].

Chapter 5
Software Requirements Specification

CHAPTER 5
SOFTWARE REQUIREMENT SPECIFICATIONS

5.1 FUNCTIONAL REQUIREMENTS

The **functional requirements** define the core operations and features that the **Digital Credit Transparency System (Credit Trust)** must perform to achieve its objectives. These requirements describe how the system interacts with shopkeepers and customers to manage credit (udhari) records efficiently. They also explain how the system processes transactions, tracks payments, and stores data securely using cloud technology. Additionally, they ensure transparency, real-time updates, and smooth functioning of all credit management activities.

- **User Authentication:** The system should provide a secure registration and login mechanism for shopkeepers using unique credentials such as username and password. It must ensure data privacy by restricting unauthorized access and protecting sensitive financial records. Proper authentication also helps in maintaining accountability and secure usage of the system.
- **Customer Management:** The system should allow shopkeepers to easily add, update, and delete customer details including name, contact number, and address. This feature helps in maintaining an organized database of customers, making it easier to manage and track individual credit records efficiently.
- **Credit Transaction Recording:** The system must enable shopkeepers to record all udhari (credit) transactions with complete details such as amount, date, and description of the purchase. This ensures accurate documentation of every transaction and helps in avoiding confusion or disputes later.
- **Real-Time Data Updates:** All data in the system should be updated instantly whenever a transaction is made or modified. Real-time updates ensure that the information displayed is always accurate and consistent across the platform for both shopkeepers and customers.
- **Customer Access:** The system should allow customers to view their credit history and transaction details through the application. This improves transparency, builds trust, and allows customers to verify their records and report any discrepancies if found.
- **Data Storage:** The system must securely store all customer data, credit records, and billing information in a cloud database such as Firebase. This ensures data safety, prevents loss due to physical damage.

- **Error Handling:** The system should include proper validation mechanisms to prevent incorrect or duplicate data entries. It should also display error messages for invalid inputs, ensuring data accuracy and improving overall system reliability.
- **Report Generation:** The system should generate detailed reports and summaries of credit records, pending payments, and transaction history. These reports help shopkeepers analyze their business performance, make better financial decisions, and maintain proper records.
- **Payment Tracking:** The system should track all payments made by customers and automatically update their remaining balance. This feature helps shopkeepers monitor pending dues, identify defaulters, and maintain a clear record of completed and pending transactions.

5.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes, operational standards, and system constraints that ensure the Digital Credit Transparency System (Credit Trust) operates efficiently, securely, and reliably. These requirements focus on enhancing system performance, usability, scalability, and data protection to provide a smooth and trustworthy experience for both shopkeepers and customers

- **Performance:** The system should deliver fast and efficient performance while recording credit (udhari) transactions, updating payment details, and retrieving customer records. It must be capable of handling multiple operations simultaneously without lag or delay, ensuring smooth functioning even during peak usage hours. Quick response time is essential to support real-time updates and improve user productivity.
- **Reliability:** The system should be highly reliable and capable of functioning continuously without crashes, system failures, or unexpected interruptions. It must ensure that all credit records, customer information, and transaction data are accurately stored and retrieved without any loss or corruption. In case of network issues, server problems, or high usage, the system should still maintain stability and recover gracefully. Reliable performance builds user confidence and ensures that shopkeepers can depend on the system for critical financial data.
- **Usability:** The application should be designed with a simple, clean, and intuitive interface that is easy to understand for both technical and non-technical users. The navigation should be straightforward, with clearly labeled buttons, forms, and instructions to guide users through different features. The system should minimize complexity and provide a smooth

user experience so that shopkeepers can quickly learn how to use it without requiring training. Good usability increases user satisfaction and encourages adoption of the digital system over traditional methods.

- **Scalability**: The platform should be capable of scaling up to accommodate an increasing number of users, interviews, and data without affecting performance. Cloud integration (AWS, Azure, or Google Cloud) can be used for scalable deployment.
- **Security:** The system must implement strong security measures to protect sensitive customer and financial data from unauthorized access, breaches, or misuse. This includes secure login authentication, encrypted data storage, and safe communication protocols such as HTTPS. Access control mechanisms should ensure that only authorized users can view or modify data. Regular security updates and monitoring should be in place to safeguard the system against potential threats, ensuring trust and data integrity.
- **Availability:** The system should be available for use at all times (24/7) with minimal downtime, allowing shopkeepers to access their records whenever needed. It should be designed with proper server management, backup systems, and recovery mechanisms to handle unexpected failures or maintenance issues. High availability ensures uninterrupted service and supports continuous business operations without disruptions.
- **Data Privacy and Compliance:** The system should ensure that all user data is handled with strict confidentiality and in accordance with data protection standards. Personal and financial information of customers must not be shared or misused under any circumstances. The platform should follow ethical data handling practices and allow secure storage and management of sensitive information. Maintaining data privacy builds trust among users and ensures responsible use of digital technology.

Chapter 6
Software Design

CHAPTER 6
SOFTWARE DESIGN

6.1 OVERVIEW

This chapter outlines the software design for the **"Credit Trust: A Digital Credit Transparency System for Local Retail Shops"** project.

6.2 USE CASE DIAGRAM

The **Use Case Diagram** for the **Credit Trust: Digital Credit Transparency System for Local Retail Shops** shows the interaction between shopkeepers, customers, and the system. It illustrates how shopkeepers manage customers, record credit transactions, and generate reports. Customers can view their credit records, receive notifications, and raise disputes. The system ensures secure data handling, real-time updates, and transparency in credit management.

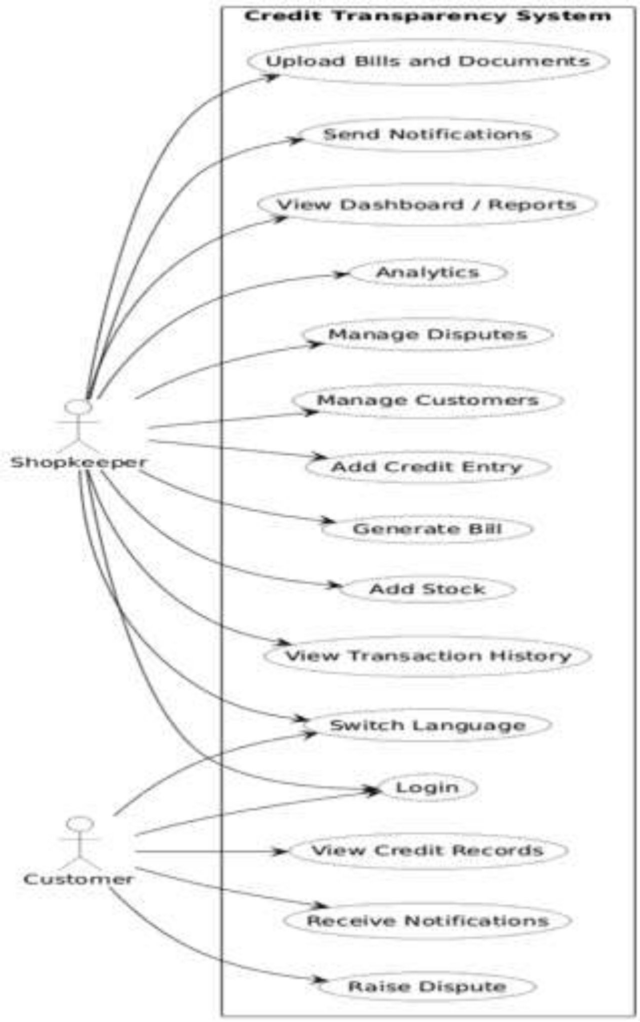

Figure 6.1: Use Case Diagram

6.3 PROJECT FLOW DIAGRAM

The system starts with a welcome screen where users select their preferred language, followed by login or signup as either an admin or customer. After authentication, users are directed to their respective dashboards, where the admin manages customers, stock, billing, and records payments through Jama Credit. The system automatically updates outstanding dues and provides analytics and reports for better financial tracking. Customers can view their transactions, credit records, and raise disputes if required. This complete flow ensures smooth operation, accurate record management, and transparency in digital credit handling.

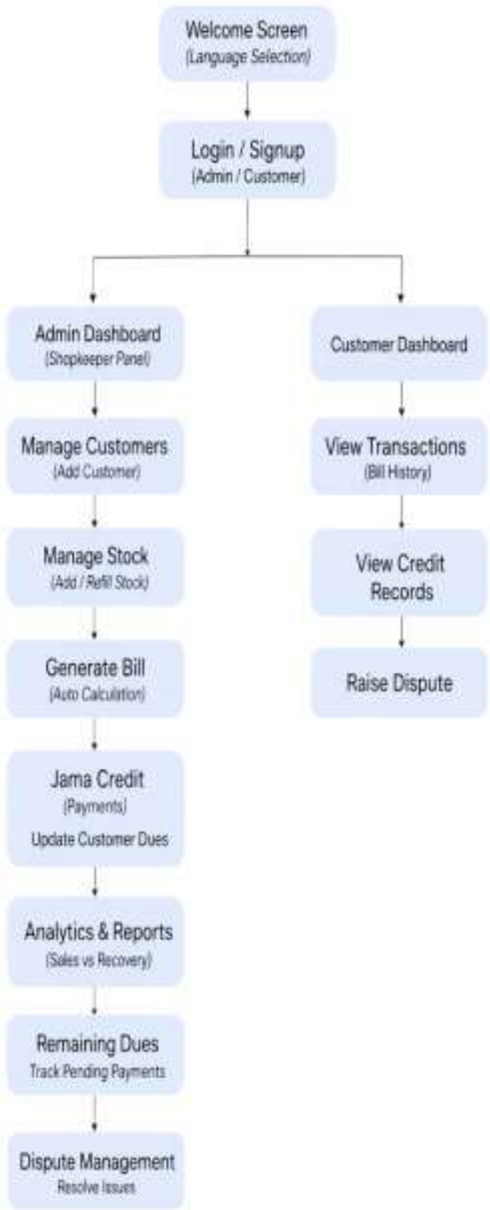

Figure 6.2: Project Flow Diagram

Chapter 7
Machine Learning Module

CHAPTER 7
MACHINE LEARNING MODULE

This chapter provides an overview of the intelligent modules implemented in the “Credit Trust” system. It aims to explain the workflow and techniques used to transform raw financial and transactional data into meaningful insights, enabling efficient credit management and improved decision-making for shopkeepers.

7.1 VISUAL INTELLIGENCE SYSTEM

The system transforms raw financial data into meaningful insights using interactive charts and graphical representations.

7.1.1 VISUALIZATION TOOLS

- **Bar Charts**: Compare total credit given versus recovered amount.
- **Line Graphs**: Display recovery trends over the last six months.

7.2 AUTOMATED VOICE SUMMARY SYSTEM

The Automated Voice Summary System enhances accessibility and usability for shopkeepers by providing audio-based summaries of financial data. This Voice Intelligence Module is built using Flutter TTS (Text-to-Speech engine), enabling the system to convert text data into clear voice outputs. It supports bilingual functionality, automatically delivering summaries in both Hindi and English, such as “Aaj ka total udhaar ₹[amount] hai...” and “Today’s total pending dues are ₹[amount]...”. The system incorporates automated scheduling using local storage to track the last playback, ensuring that the summary is played only once per day, specifically on the first admin login. Additionally, it provides interactive controls that allow the admin to play, pause, and restart voice summaries at any time, making the system more flexible and user-friendly.

Chapter 8
Coding

CHAPTER 8
CODING

8.1 ADMIN MODULE

Dashboard.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'app_colors.dart';
import 'app_strings.dart';
import 'drawer_menu.dart';
import 'home_page.dart';
import 'add_customer_page.dart';
import 'add_stock_page.dart';
import 'generate_bill_page.dart';
import 'jama_credit_page.dart';
import 'remaining_due_page.dart';
class DashboardPage extends StatefulWidget {
  const DashboardPage({super.key})
  @override
  State<DashboardPage> createState() => _DashboardPageState();
}

class _DashboardPageState extends State<DashboardPage> {
  final User? currentUser = FirebaseAuth.instance.currentUser;
  String _username = 'Admin';
  bool _isLoading = true;
  int _selectedIndex = 0;
  List<Widget> get _pages => [
    const PendingDuesPage(),    // 0
    const AddCustomerPage(),    // 1
    const AddStockPage(),       // 2
    const GenerateBillPage(),   // 3
```

```
  const JamaCreditPage(),     // 4
  const RemainingDuePage(),   // 5
  const AnalyticsPage(),      // 6
  const ReminderPage(),       // 7
  const ProfilePage(),        // 8
  const ChangePasswordPage(), // 9
  const DisputePage(),        // 10
  const DocumentLibraryPage(), // 11
  RemainingStockPage(onNavigateToAddStock: (idx) {
    setState(() {
      _selectedIndex = idx;
    });
  }), // 12
];
@override
void initState() {
  super.initState();
  _fetchUserData();
}
Future<void> _fetchUserData() async {
  if (currentUser != null) {
    try {
      DocumentSnapshot userDoc = await FirebaseFirestore.instance
          .collection('users')
          .doc(currentUser!.uid)
          .get();
      if (userDoc.exists && userDoc.data() != null) {
        final data = userDoc.data() as Map<String, dynamic>;
        setState(() {
          _username = data['name'] ?? currentUser!.email?.split('@')[0] ?? 'Admin';
        });
      }
    } catch (e) {
      debugPrint("Error fetching user data: \$e");
```

```
    }
  }
  setState(() {
    _isLoading = false;
  });
}
                              Text(
                                AppStrings.isHindi
                                    ? 'स्वागत है, $_username!'
                                    : 'Welcome, $_username!',
                                style: const TextStyle(
                                  fontSize: 18,
                                  fontWeight: FontWeight.bold,
                                  color: AppColors.textPrimary,
                                ),
                              ),
                              Text(
                                AppStrings.isHindi
                                    ? 'अपनी दुकान कुशलतापूर्वक प्रबंधित करें'
                                    : 'Manage your shop efficiently',
                                style: const TextStyle(
                                  fontSize: 14,
                                  color: AppColors.textSecondary,
                                ),
                              ),
                            ],
                          ),
                        ),
                      ],
                    ),
                  ),
                ),
              Expanded(
```

```
        child: _pages[_selectedIndex],
      ),
    ],
}
```

GenerateBillPage.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:file_picker/file_picker.dart';
import 'app_colors.dart';
import 'firebase_service.dart';
import 'notification_service.dart';
import 'mongodb_service.dart';
class GenerateBillPage extends StatefulWidget {
  const GenerateBillPage({super.key});
  @override
  State<GenerateBillPage> createState() => _GenerateBillPageState();
}
class _GenerateBillPageState extends State<GenerateBillPage> {
  String? _selectedCustomerId;
  String? _selectedCustomerPhone;
  String? _selectedCustomerEmail;
  String? _selectedCustomerName;
  // New: List of items in the current bill
  final List<Map<String, dynamic>> _items = [];

  // Controllers for the "current item" being added
  final _productController = TextEditingController();
  final _quantityController = TextEditingController();
  final _priceController = TextEditingController();
  double _itemNetQty = 1;
  String _itemUnit = 'unit';
  final _paidController = TextEditingController();
```

```
final _remainingController = TextEditingController();
final _noteController = TextEditingController();
bool _isSaving = false;
// Document attachments
final List<String> _attachedFileNames = [];

final FirebaseService _firebaseService = FirebaseService();

double get _totalAmount => _items.fold(0, (sum, item) => sum + (item['total'] as double));

void _addItem() {
  final String product = _productController.text.trim();
  final double qty = double.tryParse(_quantityController.text) ?? 0;
  final double price = double.tryParse(_priceController.text) ?? 0;

  if (product.isEmpty || qty <= 0 || price <= 0) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Please enter valid product name, quantity, and price')),
    );
    return;
  }

  setState(() {
    _items.add({
      'productName': product,
      'quantity': qty,
      'price': price,
      'netQuantity': _itemNetQty,
      'unit': _itemUnit,
      'total': qty * price,
    });
    _productController.clear();
    _quantityController.clear();
    _priceController.clear();
```

```
    _itemNetQty = 1;
    _itemUnit = 'unit';
    _calculateRemaining();
  });
}

void _removeItem(int index) {
  setState(() {
    _items.removeAt(index);
    _calculateRemaining();
  });
}

void _calculateRemaining() {
  double total = _totalAmount;
  double paid = double.tryParse(_paidController.text) ?? 0;
  _remainingController.text = (total - paid).toStringAsFixed(2);
}

Future<void> _generateBill() async {
  if (_selectedCustomerId == null || _items.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Please select a customer and add at least one item')),
    );
    return;
  }

  setState(() => _isSaving = true);

  try {
    final double totalAmount = _totalAmount;
    final double totalPaid = double.tryParse(_paidController.text) ?? 0.0;
    final double remainingAmount = totalAmount - totalPaid;
```

```
final billData = {
  'customerId': _selectedCustomerId,
  'items': _items,
  'totalAmount': totalAmount,
  'totalPaid': totalPaid,
  'remainingAmount': remainingAmount,
  'note': _noteController.text.trim(),
  'attachments': _attachedFileNames,
};

final double totalBalance = await _firebaseService.generateBill(_selectedCustomerId!, billData, totalAmount, remainingAmount);

// Trigger Notifications
if (_selectedCustomerPhone != null) {
  await NotificationService.sendBillNotification(
    phone: _selectedCustomerPhone!,
    items: _items,
    totalBill: totalAmount,
    paid: totalPaid,
    remainingDue: remainingAmount,
    totalOutstandingBalance: totalBalance,
  );
}

if (_selectedCustomerEmail != null) {
  await NotificationService.sendEmailNotification(
    email: _selectedCustomerEmail!,
    customerName: _selectedCustomerName ?? 'Customer',
    items: _items,
    totalAmount: totalAmount,
    paidAmount: totalPaid,
    remainingAmount: remainingAmount,
    totalOutstanding: totalBalance,
```

```
      );
    }

    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Multi-item Bill generated successfully')),
      );
      _clearFields();
    }
  } catch (e) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Error: $e')),
      );
    }
  } finally {
    if (mounted) setState(() => _isSaving = false);
  }
}

void _clearFields() {
  setState(() {
    _selectedCustomerId = null;
    _selectedCustomerPhone = null;
    _selectedCustomerEmail = null;
    _selectedCustomerName = null;
    _items.clear();
    _attachedFileNames.clear();
  });
  _productController.clear();
  _quantityController.clear();
  _priceController.clear();
  _paidController.clear();
  _remainingController.clear();
```

```
  _noteController.clear();
}

@override
Widget build(BuildContext context) {
  return SingleChildScrollView(
    padding: const EdgeInsets.all(20.0),
    child: Column(
      children: [
        // Customer Selection Card
        _buildCard(
          title: 'Customer Details',
          child: StreamBuilder<QuerySnapshot>(
            stream: _firebaseService.getCustomers(),
            builder: (context, snapshot) {
              if (!snapshot.hasData) return const CircularProgressIndicator();
              var customers = snapshot.data!.docs;
              return DropdownButtonFormField<String>(
                decoration: _inputDecoration('Select Customer', Icons.person),
                value: _selectedCustomerId,
                items: customers.map((doc) {
                  final data = doc.data() as Map<String, dynamic>;
                  return DropdownMenuItem(
                    value: doc.id,
                    child: Text(data['name'] ?? 'Unknown'),
                    onTap: () {
                      _selectedCustomerPhone = data['phone'];
                      _selectedCustomerEmail = data['email'];
                      _selectedCustomerName = data['name'];
                    },
                  );
                }).toList(),
                onChanged: (val) => setState(() => _selectedCustomerId = val),
              );
```

```
          }
        ),
      ),
      const SizedBox(height: 16),

      // Item Addition Section
      _buildCard(
        title: 'Add Products',
        child: Column(
          children: [
            LayoutBuilder(
              builder: (context, constraints) => Autocomplete<Map<String, dynamic>>(
                displayStringForOption: (option) => option['productName'],
                optionsBuilder: (textValue) async {
                  if (textValue.text.isEmpty) return [];
                  final uid = FirebaseAuth.instance.currentUser?.uid ?? '';
                  return await MongoService.searchProducts(uid, textValue.text);
                },
                onSelected: (selection) {
                  _productController.text = selection['productName'];
                  _priceController.text = (selection['price'] ?? 0).toString();
                  _itemNetQty = (selection['netQuantity'] ?? 1).toDouble();
                  _itemUnit = selection['unit'] ?? 'unit';
                },
                fieldViewBuilder: (ctx, controller, node, onSubmitted) {
                  controller.addListener(() => _productController.text = controller.text);
                  return _buildTextField('Product Name', Icons.shopping_basket, controller: controller,
focusNode: node);
                },
              ),
            ),
            const SizedBox(height: 12),
            Row(
              children: [
```

```
        // Payment + Note + Attachment Card
        _buildCard(
          title: 'Payment & Documents',
          child: Column(
            children: [
              _buildTextField('Total Paid', Icons.account_balance_wallet, controller: _paidController,
keyboardType: TextInputType.number, onChanged: (v) => _calculateRemaining()),
              const SizedBox(height: 12),
              _buildTextField('Remaining Balance', Icons.pending_actions, controller:
_remainingController, keyboardType: TextInputType.number, enabled: false),
              const SizedBox(height: 12),
              _buildTextField('Note / Remarks (optional)', Icons.sticky_note_2_outlined, controller:
_noteController),

                  onPressed: _isSaving ? null : _generateBill,
                  child: _isSaving
                    ? const CircularProgressIndicator(color: Colors.white)
                    : const Text('Confirm & Save Bill', style: TextStyle(fontSize: 16, color: Colors.white,
fontWeight: FontWeight.bold)),
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  );
 }
 InputDecoration _inputDecoration(String label, IconData icon) {
  return InputDecoration(
    filled: true,
    fillColor: AppColors.containerSoft,
    prefixIcon: Icon(icon, color: AppColors.primary),
```

```
      labelText: label,
      labelStyle: const TextStyle(color: AppColors.textSecondary),
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(12), borderSide:
BorderSide.none),
    );
  }

  Widget _buildTextField(String label, IconData icon, {TextEditingController? controller,
FocusNode? focusNode, TextInputType keyboardType = TextInputType.text, Function(String)?
onChanged, bool enabled = true}) {
    return TextField(
      controller: controller,
      focusNode: focusNode,
      onChanged: onChanged,
      enabled: enabled,
      style: TextStyle(color: enabled ? AppColors.textPrimary : AppColors.textSecondary),
      decoration: _inputDecoration(label, icon),
      keyboardType: keyboardType,
    );
  }
}
```

8.2 CUSTOMER MODULE

Dashboard.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'dart:async';
import 'app_colors.dart';
import 'login_page.dart';
import 'customer_drawer.dart';
import 'local_notification_service.dart';
```

```
class CustomerDashboardPage extends StatefulWidget {
  const CustomerDashboardPage({super.key});

  @override
  State<CustomerDashboardPage> createState() => _CustomerDashboardPageState();
}

class _CustomerDashboardPageState extends State<CustomerDashboardPage> {
  final User? currentUser = FirebaseAuth.instance.currentUser;
  String _customerName = 'Customer';
  String _shopName = 'Your Shop';
  String _adminUid = '';
  bool _isLoading = true;
  int _selectedIndex = 0;
  StreamSubscription? _billSubscription;
  int _lastBillCount = -1; // -1 means not yet initialized

  @override
  void initState() {
    super.initState();
    _fetchCustomerProfile();
    // Request notification permission as soon as the customer opens the app
    LocalNotificationService.initialize();
  }

  Future<void> _fetchCustomerProfile() async {
    if (currentUser == null) return;

    try {
      final docRef = FirebaseFirestore.instance
          .collection('customer_accounts')
          .doc(currentUser!.uid);
      final doc = await docRef.get();
```

```
    if (doc.exists) {
      final data = doc.data() as Map<String, dynamic>;
      setState(() {
        _customerName = data['name'] ?? 'Customer';
        _shopName = data['shopName'] ?? 'Your Shop';
        _adminUid = data['adminUid'] ?? '';
      });
      _startBillListener(); // Start listening for new bills
    }
  } catch (e) {
    debugPrint("Error fetching customer profile: $e");
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}

void _startBillListener() {
  if (currentUser == null) return;
  _billSubscription?.cancel();
  _billSubscription = FirebaseFirestore.instance
      .collection('customer_accounts')

@override
Widget build(BuildContext context) {
  if (_isLoading) {
    return const Scaffold(
      body: Center(child: CircularProgressIndicator(color: AppColors.primary)),
    );
  }

  return Scaffold(
    backgroundColor: AppColors.screenBg,
    appBar: AppBar(
      title: Text(
```

```
        ['Customer Dashboard', 'Bill History', 'Credit Record', 'Raise
Dispute'][_selectedIndex],
        style: const TextStyle(color: AppColors.textPrimary, fontWeight: FontWeight.bold)),
      backgroundColor: Colors.white,
      elevation: 0,
      iconTheme: const IconThemeData(color: AppColors.primary),
      actions: [
        IconButton(
          icon: const Icon(Icons.notifications_outlined, color: AppColors.primary),
          tooltip: 'Notifications',
          onPressed: () => _showNotificationsPanel(context),
        ),
      ],
    ),
    drawer: CustomerDrawer(
      selectedIndex: _selectedIndex,
      customerName: _customerName,
      shopName: _shopName,
      onItemSelected: (index) {
        setState(() => _selectedIndex = index);
      },
    ),
    body: IndexedStack(
      index: _selectedIndex,
      children: [
        _buildHomeFragment(),
        _buildBillHistoryFragment(),
        _buildCreditRecordFragment(),
        _buildRaiseDisputeFragment(),
      ],
    ),
  );
}
```

```
                    }
                        ),
                        isThreeLine: true,
                        trailing: Container(
                          padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
                          decoration: BoxDecoration(
                            color: isSettled ? Colors.green.withOpacity(0.2) :
Colors.orange.withOpacity(0.2),
                            borderRadius: BorderRadius.circular(10),
                          ),
                          child: Text(
                            isSettled ? 'Settled' : 'Pending',
                            style: TextStyle(
                              color: isSettled ? Colors.greenAccent : Colors.orange,
                              fontWeight: FontWeight.bold, fontSize: 11,
                            ),
                          ),
                        ),
                      ),
                    );
                  },
                );
              },

        return ListView.separated(
          shrinkWrap: true,
          physics: limit != null ? const NeverScrollableScrollPhysics() : const                trailing:
const Icon(Icons.chevron_right, color: AppColors.textSecondary),
            ),
          );
        },
      );
    },
  );
```

```
 }

 // ── Credit Record Fragment
────────────────────────────────────────

 Widget _buildCreditRecordFragment() {
   if (currentUser == null) return const Center(child: Text('Not logged in.'));

   return StreamBuilder<QuerySnapshot>(
     stream: FirebaseFirestore.instance
         .collection('customer_accounts')
         .doc(currentUser!.uid)
         .collection('bills')
         .orderBy('createdAt', descending: true)
         .snapshots(),
     builder: (context, snapshot) {
       if (snapshot.connectionState == ConnectionState.waiting) {
         return const Center(child: CircularProgressIndicator(color: AppColors.primary));
       }

       final docs = snapshot.data?.docs ?? [];

       if (docs.isEmpty) {
         return Center(
           child: Column(
             mainAxisAlignment: MainAxisAlignment.center,
             children: [
               Icon(Icons.receipt_long_outlined, size: 70, color:
AppColors.primary.withOpacity(0.3)),
                 value: selectedCategory,
                 decoration: const InputDecoration(
                   border: InputBorder.none,
                   labelText: 'Category',
```

```
                labelStyle: TextStyle(color: AppColors.textSecondary),
              ),
              dropdownColor: AppColors.containerBg,
              style: const TextStyle(color: AppColors.textPrimary),
              items: categories.map((c) => DropdownMenuItem(value: c, child:
Text(c))).toList(),
              onChanged: (v) => setLocalState(() => selectedCategory = v ??
selectedCategory),
            ),
          ),
        ),
        const SizedBox(height: 12),

        // Reason
        Card(
          color: AppColors.containerBg,
          elevation: 2,
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
          child: Padding(
            padding: const EdgeInsets.all(12),
            child: TextField(
              controller: reasonController,
              maxLines: 5,
              style: const TextStyle(color: AppColors.textPrimary),
              decoration: InputDecoration(
                border: InputBorder.none,
                hintText: 'Describe your dispute in detail...',
                hintStyle: TextStyle(color: AppColors.textSecondary.withOpacity(0.6),
fontSize: 13),
                prefixIcon: const Padding(
                  padding: EdgeInsets.only(bottom: 60),
                  child: Icon(Icons.edit_note, color: AppColors.primary),
                ),
              ),
```

```
    ),
  ),
),
const SizedBox(height: 20),

// Submit button
SizedBox(
  width: double.infinity,
  height: 50,
  child: ElevatedButton.icon(
    style: ElevatedButton.styleFrom(
      backgroundColor: AppColors.primary,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    ),
    onPressed: isSubmitting
        ? null
        : () async {
            final reason = reasonController.text.trim();
            if (reason.isEmpty) {
              ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(content: Text('Please describe your dispute.')));
              return;
            }
            setLocalState(() => isSubmitting = true);
            await FirebaseFirestore.instance.collection('disputes').add({
              'customerId': currentUser?.uid ?? '',
              'customerName': _customerName,
              'category': selectedCategory,
              'reason': reason,
              'status': 'open',
              'createdAt': FieldValue.serverTimestamp(),
            });
            reasonController.clear();
            setLocalState(() => isSubmitting = false);
```

```
                    if (context.mounted) {
                      ScaffoldMessenger.of(context).showSnackBar(
                        const SnackBar(
                          content: Text('Dispute submitted successfully! Your shop owner will
review it.'),
                          backgroundColor: Colors.green,
                        ),
                      );
                    }
                  },
          icon: isSubmitting
              ? const SizedBox(width: 18, height: 18, child:
CircularProgressIndicator(color: Colors.white, strokeWidth: 2))
              : const Icon(Icons.send, color: Colors.white),
          label: Text(isSubmitting ? 'Submitting...' : 'Submit Dispute',
              style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold)),
        ),
      ),
    ],
  ),
 );
},
);
}
```

Chapter 9
Result and Output Screen

CHAPTER 9
RESULT AND OUTPUT SCREEN

(Figure 9.1) This is the welcome screen of the **Credit Trust app**, allowing users to select their preferred language (English or Hindi). It provides a simple and user-friendly start to access the digital credit management system.

Figure 9.1: Welcome Page

(Figure 9.2) This is the login page where users can sign in as **Admin** using their email and password. It provides secure access to the Credit Trust system with options for password recovery and new user registration.

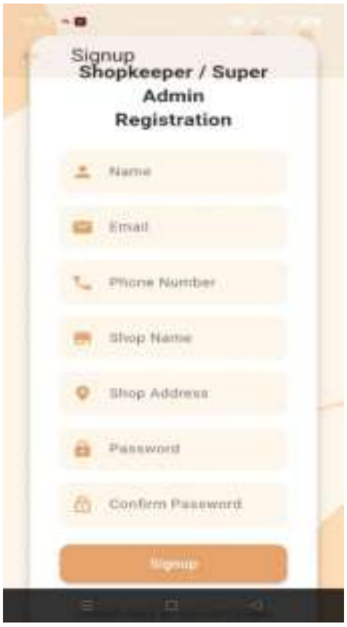

Figure 9.2: Signup Page

(Figure 9.3) The Login interface of Credit Trust provides returning users/admin with a secure and user-friendly way to sign in using their email and password.

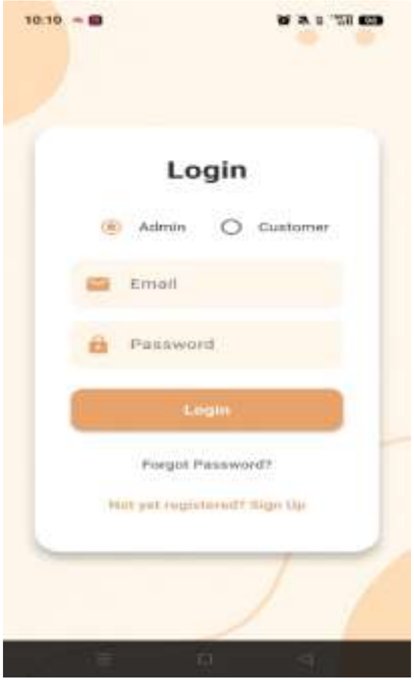

Figure 9.3: Login Page

(Figure 9.4) The Shopkeeper Dashboard interface of Credit Trust welcomes the user personally and provides an overview of the shop's financial and operational status. It displays key information such as total pending dues, number of customers with outstanding balances, daily sales, and stock availability, offering an efficient and user-friendly environment for shopkeepers to track and manage their credit transactions and inventory effectively.

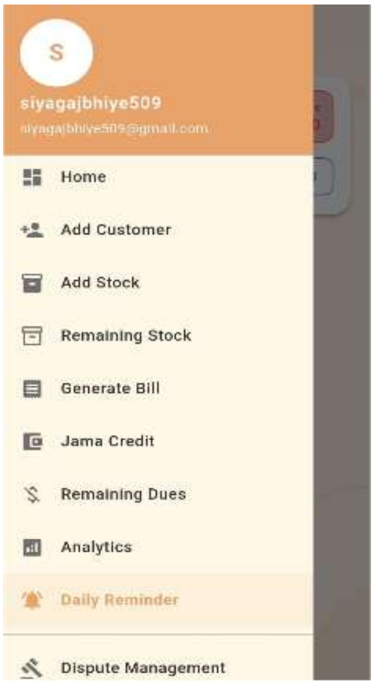

Figure 9.4: Admin Dashboard Page

(Figure 9.5) The Admin Dashboard interface of Credit Trust displays pending dues, recent transactions, and customer-wise balances in a clear layout. It also includes a voice feature that announces the total remaining dues when the admin logs in for the first time each day, ensuring quick and convenient updates.

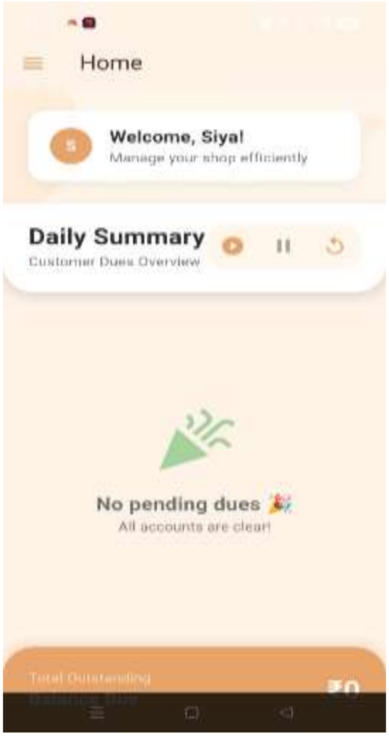

Figure 9.5: Home Interface of Admin Dashboard

(Figure 9.6) The Add Customer interface of Credit Trust enables the admin to create and manage customer profiles by entering essential details such as name, contact information providing a simple and efficient way to maintain customer credit records.

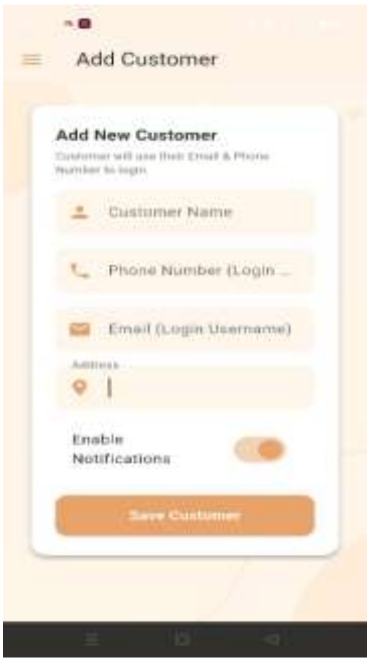

Figure 9.6: Add Customer Interface

(Figure 9.7) The Add Stock interface of Credit Trust allows the admin to manage inventory by adding product details, quantity, and pricing information, enabling automatic bill generation and efficient stock refilling for smooth shop operations.

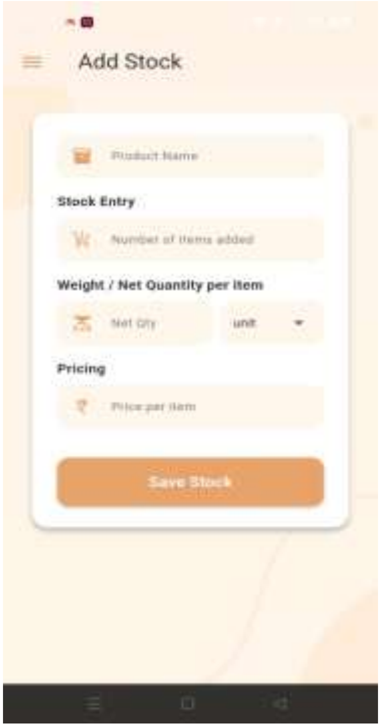

Figure 9.7: Add Stock Interface

(Figure 9.8) The Refill Stock interface of Credit Trust enables the admin to update existing product quantities by adding new stock, ensuring accurate inventory management and smooth continuation of billing operations.

Figure 9.8: Refill Stock Interface

(Figure 9.9) The Bill Generation interface of Credit Trust enables the admin to create bills by adding multiple items, automatically calculating the total amount and recording the paid amount, while transferring the remaining balance to customer credit for efficient transaction management.

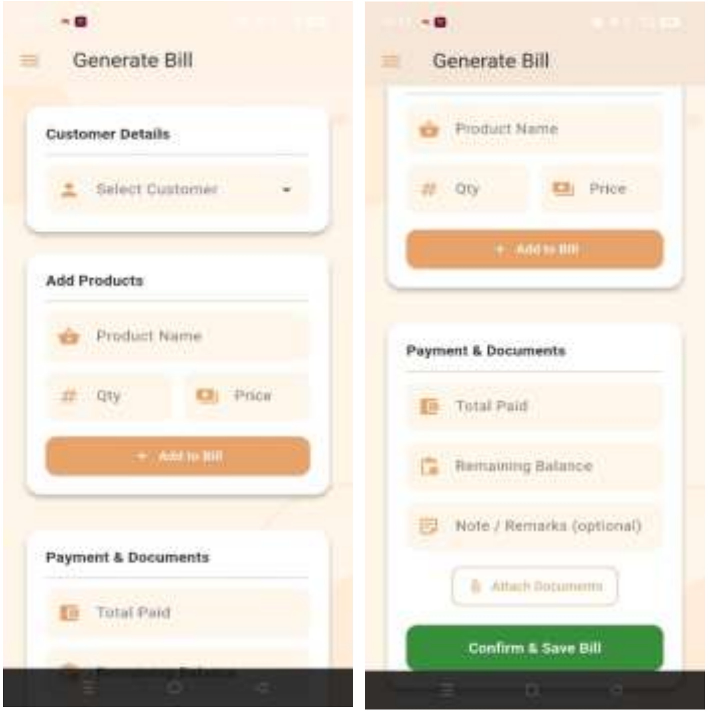

Figure 9.9: Bill Generation Interface

(Figure 9.10) The Jama Credit interface of Credit Trust enables the admin to record customer payments by updating the paid amount and reducing the outstanding credit balance, ensuring accurate tracking and management of customer dues.

Figure 9.10: Jama Credit Interface

(Figure 9.11) The Analytics interface of Credit Trust provides insights into sales and recovery data by comparing total sales with collected payments, enabling the admin to monitor financial performance and track outstanding dues effectively.

Figure 9.11: Analytics Interface

(Figure 9.12) The Remaining Dues interface of Credit Trust displays all pending customer balances in a clear and organized layout, enabling the admin to easily track outstanding payments and manage credit efficiently

Figure 9.12: Remaining Dues Interface

(Figure 9.13) The Dispute Management interface of Credit Trust enables the admin to handle and resolve discrepancies in transactions and customer dues, ensuring transparency and accuracy in credit records.

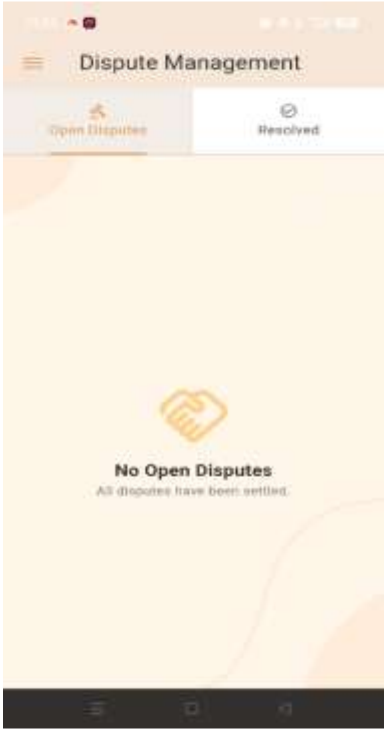

Figure 9.13: Dispute Management Interface

(Figure 9.14) The Customer Dashboard interface of Credit Trust provides customers with a clear view of their transactions, outstanding dues, and payment history, enabling them to track their credit status and manage payments transparently.

Figure 9.14: Customer Dashboard

(Figure 9.15) The Bill History interface of Credit Trust displays a record of all past transactions, including billed items, total amounts, and payment details, enabling customers to track their purchase history and outstanding dues effectively.

Figure 9.15: Bill History Interface

(Figure 9.16) The Credit Record interface of Credit Trust displays a detailed history of customer credit transactions, including dues, payments, and remaining balances, enabling users to track and manage their credit records effectively.

Figure 9.16: Credit Record Interface

(Figure 9.17) The Raise Dispute interface of Credit Trust allows customers to report issues related to transactions or dues by submitting their concerns, enabling effective resolution and maintaining transparency in credit records.

Figure 9.17: Raise Dispute Interface

Chapter 10
Conclusion and Future Work

CHAPTER 10
CONCLUSION & FUTURE WORK

10.1 Conclusion

The Credit Trust application successfully provides a digital solution for managing credit transactions and inventory in small kirana and general stores. By integrating features such as customer credit tracking, automated bill generation, stock management, and real-time analytics, the system ensures accurate and efficient handling of daily shop operations.The application simplifies traditional udhaar (credit) systems by maintaining transparent records of dues, payments, and transaction history. Additional features like voice-assisted summaries and dispute management further enhance usability and accessibility for shopkeepers. The system also enables better decision-making through insights into sales and recovery patterns. Overall, Credit Trust bridges the gap between manual record-keeping and modern digital solutions, empowering small business owners to manage their finances, track customer credit, and maintain inventory more effectively and efficiently.

10.2 Future Work

- **AI-Based Credit Analysis:** Integrate AI models to analyze customer payment behavior and predict credit risk, helping shopkeepers make informed decisions while giving credit.
- **Integration with Digital Payments:** Connect the system with UPI and other digital payment platforms to enable automatic updates of payments and reduce manual entry.
- **Mobile Application Enhancements:** Improve the mobile experience with faster performance, offline support, and real-time synchronization for seamless usage.
- **Automated Reminders:** Implement WhatsApp/SMS notifications to remind customers about pending dues and improve payment recovery.
- **Barcode and QR Integration:** Add barcode scanning and QR-based billing to speed up product entry and billing processes.

By advancing these areas, the Credit Trust system can evolve into a comprehensive and intelligent solution for credit management, inventory tracking, and business analytics, providing greater efficiency, transparency, and scalability for small retail businesses.

REFERENCES

JOURNALS / RESEARCH PAPERS

1. Research on Digital Transformation of Kirana Stores
   https://www.researchgate.net/publication/400100801_KIRANA_40_Measuring_the_Impact_of_Phygital_Transformation_of_Small_Retailers_on_BOP_Consumer_Behaviour
   This research shows that digital systems improve customer trust, satisfaction, and overall retail efficiency in kirana stores.
2. Andrew Stephen (2015), The Role of Digital and Social Media Marketing in Consumer Behaviour, Current Opinion in Psychology special issue on Consumer Behaviour, 1-16 .
3. Babu Sudhir D and Narayanamma Lakshmi P 2018 Consumer Perception Towards Digital Payment International Journal of Emerging Technologies and Innovative Research ISSN at http://www.jetir.org/papers/JETIRA006063.pdf
4. Alkhowaiter W A 2020 Digital payment and banking adoption research in Gulf countries A systematic literature review International Journal of Information Management Volume 53 Page 102102.

PROJECT SUMMARY

About Project

Title of the project	**Credit Trust: A Digital Credit Transparency System for Local Retail Shops**
Semester	**8th**
Members	**1. Shruti Tiwari** **2. Siya Gajbhiye** **3. Sitvat Fatima**
Team Leader	**Shruti Tiwari**
Describe role of every member in the project	**Shruti Tiwari:** Flutter app develops **Siya Gajbhiye:** Customer module, visualization module **Sitvat Fatima:** Firebase, and Mongo DB integration
What is the motivation for selecting this project?	**The motivation for this project is to solve problems of manual credit (udhari) record management in local shops. It aims to provide a simple digital solution for better accuracy and transparency. It also supports digital growth of small businesses under SDG 8.**
Project Type **(Desktop Application, Web Application, Mobile App, Web)**	**Mobile Application**

Tools & Technologies

Programming language used	**Dart**
Compiler used **(with version)**	**NA**
IDE used **(with version)**	**Microsoft Visual Studios Code (1.51.1)**
Front End Technologies **(with version, wherever Applicable)**	**Flutter**
Back End Technologies **(with version, wherever applicable)**	**Firebase**

Software Design& Coding

Is prototype of the software developed?	**NA**
SDLC model followed **(Waterfall, Agile, Spiral etc.)**	**Agile Methodology**
Why above SDLC model is followed?	**Agile model has a set of guidelines that are: small, highly motivated project team and supports changing requirements. We need both guidelines to develop our project.**
Justify that the SDLC model mentioned above is followed in the project.	**Since the demand (functionalities) of the website kept on changing every now and then therefore we used the Agile model, so that we could make desired changes whenever needed.**
Software Design approach followed **(Functional or ObjectOriented)**	**Object oriented approach**
Name the diagrams developed **(according to the Design approach followed)**	**Use Case Diagram and Project Flow Diagram**
In case Object Oriented approach is followed, which of the OOPS principles are covered in design?	**1. Encapsulation** **2. Inheritance** **3. Polymorphism** **4. Abstraction**
No. of Tiers **(example 3-tier)**	**3-tier**
Total no. of front end pages	**17**
Total no. of tables in database	**1**
Database is in which Normal Form?	**3 NF**
Are the entries in database encrypted?	**Yes**
Front end validations applied **(Yes / No)**	**Yes**
Session management done **(in case of web applications)**	**NA**
Is application browser compatible **(in case of web applications)**	**Yes**

Exception handling done **(Yes / No)**	**Yes**
Commenting done in code **(Yes / No)**	**Yes**
Naming convention followed **(Yes / No)**	**Yes**
What difficulties faced during deployment of project?	**Challenges included collecting and preparing a suitable dataset, selecting the most accurate model through experimentation, and integrating it effectively**
Total no. of Use-cases	**15**
Give titles of Use-cases	**1. Upload Bills and Documents** **2. Send Notifications** **3. View Dashboard/Reports** **4. Analytics** **5. Manage Dispute** **6. Manage Customer** **7. Add Credit Entry** **8. Generate Bill** **9. Add Stock** **10. View Transaction History** **11. Switch Language** **12. Login** **13. View Credit Records** **14. Receive Notification** **15. Raise Dispute**

Project Requirements

MVC architecture followed **(Yes / No)**	**Yes**
If yes, write the name of MVC architecture followed **(MVC-1, MVC-2)**	**MVC-2**
Design Pattern used **(Yes / No)**	**No**
If yes, write the name of Design Pattern used	**No**
Interface type **(CLI / GUI)**	**GUI**
No. of Actors	**2**
Name of Actors	**Shopkeeper/Admin , Customer**
Total no. of Functional Requirements	**9**
List few important non-Functional Requirements	**Performance, Reliability, Usability, Scalability, Security, Availability, Data Privacy and Compliance**

Testing

Which testing is performed? **(Manual or Automation)**	**Manual**
Is Beta testing done for this project?	**No**

Write project narrative covering above mentioned points

The project **"Credit Trust: A Digital Credit Transparency System for Local Retail Shops"** addresses issues of manual credit (udhari) management in local businesses. Paper-based systems often cause errors, data loss, and lack of transparency. This system provides a mobile app to manage customers, record transactions, and track payments digitally. It uses Firebase for secure storage and real-time updates. The solution improves efficiency, trust, and supports digital growth under **SDG 8**.

Shruti Tiwari 0187CS221189
Siya Gajbhiye 0187CS221192
Sitvat Fatima 0187CS221191

Guide Signature
(Dr. Komal Tahiliani)

APPENDIX GLOSSARY OF TERMS

(In alphabetical order)

D

Database A structured system is used to store, manage, and retrieve data efficiently. In this project, databases like Firestore and MongoDB are used to store customer records, credit transactions, and billing details securely.

F

Firebase A cloud-based backend platform by Google that provides services like authentication, real-time database, and hosting. It is used in this project for secure data storage and real-time updates.

Firestore A NoSQL cloud database provided by Firebase that stores data in collections and documents. It allows real-time synchronization and efficient data handling for the application.

Flutter An open-source UI toolkit developed by Google used to build cross-platform mobile applications. It enables fast development with a single codebase for Android and iOS.

.

I

IDE A software application that provides tools for software development, including a code editor, debugger, and compiler. It simplifies the process of writing, testing, and debugging code.

M

MongoDB ASQL database that stores data in a flexible, JSON-like format. It is used for managing large amounts of unstructured or semi-structured data efficiently.

U

UI The space where interactions between humans and machines occur. It includes the design and layout of elements like buttons, menus, and screens that allow users to interact with software or hardware in a way that is intuitive and efficient.

V

VS CODE A lightweight and powerful code editor by Microsoft that supports multiple programming languages, extensions, debugging, and version control integration for efficient software development.